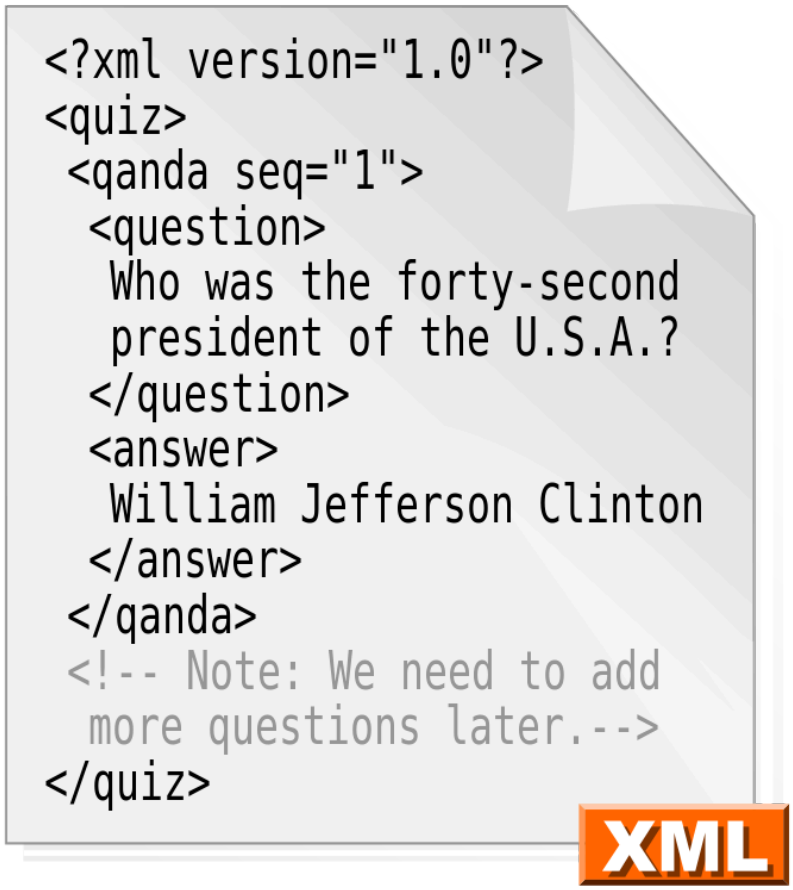
Definición de esquemas y vocabularios en xml.

Hasta ahora hemos trabajado con documentos básicos de XML y no hemos analizado en profundidad cómo se elaboran y algunos de los aspectos que lo popularizó para muchísimos usos: realizar búsqueda inteligente e intercambio de datos, la presentación adaptable y la personalización de datos, permiten a nuevas aplicaciones buscar, mover, visualizar y manipular la información más fácilmente y permite automatizar procesos.

Como veis, lo relevante de este metalenguaje son los datos. El XML es un lenguaje de marcas de estructuración de datos, separa los datos de su presentación (pero no hace nada por sí mismo, en comparación con el HTML). Para comprender su relevancia hay que ahondar en su sintaxis y en su construcción con respecto a modelos que puedan facilitar a las aplicaciones todas esas utilidades que acabamos de mencionar.



1.- Documento XML. Estructura y sintaxis.

La sintaxis será un conjunto de reglas para que pueda ser entendido por todas las aplicaciones y usuarios. Cuanto más estrictas sean estas normas más fáciles serán las aplicaciones que trabajen con él.



Un archivo XML es un archivo de texto que contiene un determinado número de etiquetas estructuradas en árbol.Esos elementos estarán vinculados y al igual que en HTML serán … . En XML todas las etiquetas deberán cerrarse y como ya se mencionó, cualquier palabra puede ser una etiqueta. A diferencia del HTML, no existe un conjunto limitado de etiquetas, de ahí lo de eXtensible.

En la primera unidad vimos que un documento XML básico estaba formado por un prólogo y un ejemplar o cuerpo. Recordamos que cada una de esas partes tiene el siguiente cometido:

PARTES	COMETIDO
Prólogo	Informa al intérprete encargado de procesar el documento de todos aquellos datos que necesita para realizar su trabajo. Consta de dos partes: • **Definición de XML:** Donde se indica la versión de XML que se utiliza, el código de los datos a procesar y la autonomía del documento. Este último dato hasta ahora siempre ha sido "yes" ya que los documentos generados eran independientes. • **Declaración del tipo de documento (DTD):** Hasta el momento solo hemos dicho que es el nombre del ejemplar precedido de la cadena
Ejemplar o cuerpo	Contiene los datos del documento que se quiere procesar. **Es el elemento raíz del documento y ha de ser único**. Está compuesto de elementos estructurados según una estructura de árbol en la que el elemento raíz es el ejemplar y las hojas los elementos terminales, es decir, aquellos que no contienen elementos. Los elementos pueden estar a su vez formados por atributos.

Para saber más...

Existe una versión 1.1 del estándar XML que incorpora algunas variaciones (fundamentalmente para adaptarlas a las evoluciones del Unicode 3.0 que añade nuevas escrituras: mongol, camboyano y birmano), si no se utilizan (como suponemos que va a ser nuestro caso), el W3C recomienda utilizar la 1.0, por lo que, de momento siempre utilizaremos el estándar XML 1.0 en nuestro prólogo de todos los documentos XML.

Autoevaluación

Marcar los componentes de un documento XML:

- [ ] Prólogo.
- [ ] Ejemplar.
- [ ] Definición de codificación del documento.
- [ ] Cabecera.

Mostrar Información

1.1.- Declaración de tipo de documento (DTD)

¿Qué es? Una DTD declara las normas de validez de un documento XML. Define el orden de aparición y de imbricación de los elementos, atributos y sus tipos, así como las restricciones del documento, valores por defecto, etc. Podrán ser internas (poco utilizadas) o externas como un archivo de texto plano con extensión .dtd.

La declaración del tipo de documento comienza con el texto que indica el nombre del tipo. El nombre del tipo ha de ser idéntico al del ejemplar (o elemento raíz) del documento XML en el que se está trabajando.

¿Cómo incluiremos la referencia de la dtd externa en un documento XML? En el prólogo del documento después de la versión XML:

ó

SYSTEM y PUBLIC significa que la dtd es de uso privado por una organización o pública.

En la imagen del ejemplo se puede ver un documento XML con elemento raíz person que estará definida por la DTD person.dtd (en ese archivo, como explicaremos a continuación se define cómo debe ser el elemento person, con un elemento name que a su vez tiene first_name y last_name y profession, en ese orden).

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE person SYSTEM "person.dtd">
<person>
  <name>
    <first_name>David</first_name>
    <last_name>Perez</last_name>
  </name>
  <profession>Teacher</profession>

</person>
```

Autoevaluación

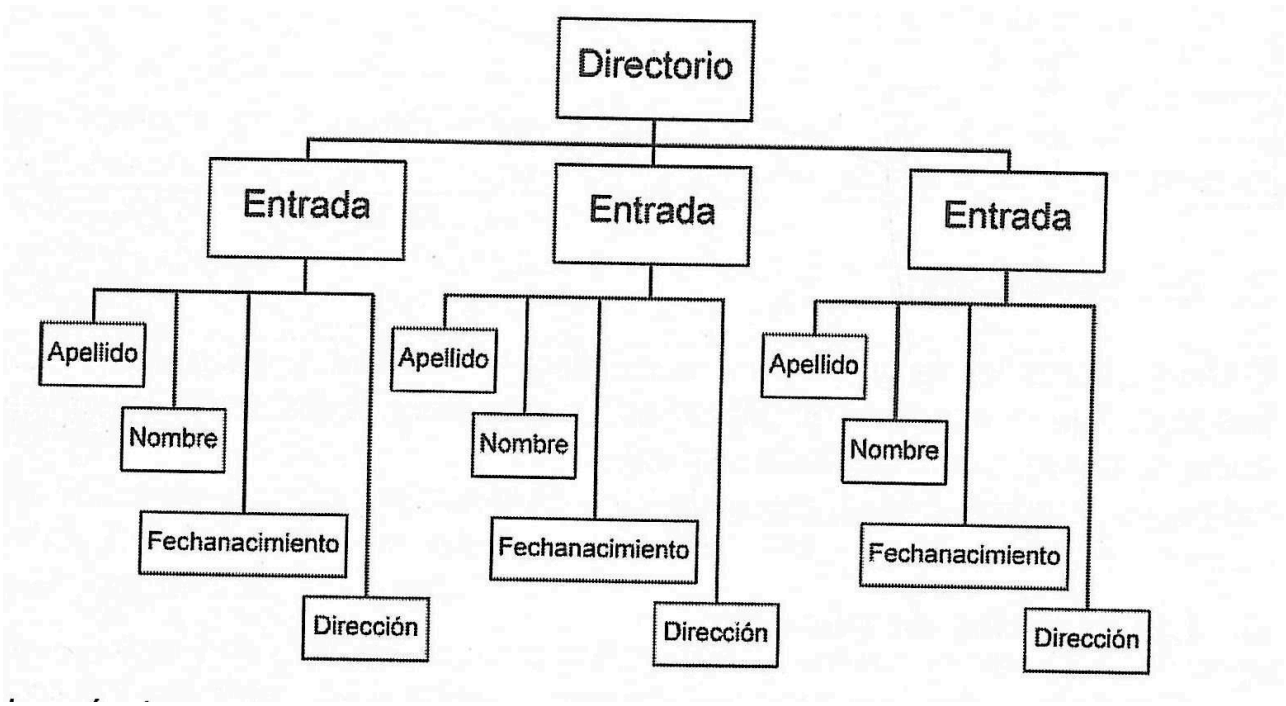
La definición de tipo de documento ha de ser:

- ○ Interna o externa al documento XML al que está referida.
- ○ Interna al documento XML que le refiere.
- ○ Externa al documento XML que le refiere.
- ○ Única.

1.2.- Definición de la sintaxis de documentos XML.

Recordamos que en estos documentos las etiquetas de marcado describen la estructura del documento.

> Un **elemento** es un componente lógico de un documento que o bien comienza por una etiqueta de apertura con atributos si los tuviera y termina por la etiqueta de cierre correspondiente o que consiste en una única etiqueta vacía. El contenido de un elemento es todo lo que se encuentra entre las etiquetas de apertura y cierre, incluso si estos son también elementos en cuyo caso se llaman elementos hijos.



Pasaremos a hacer algunas consideraciones sobre cada uno de ellos: etiquetas, atributos y contenidos.

Etiquetas o nombres de los elementos

Algunas consideraciones sobre los nombres de los elementos o etiquetas:

- Delimitadas por símbolos **<** y **>**
- Respetar mayúsculas y minúsculas: los nombres para etiqueta de apertura y de cierre han de ser idénticos. (Recomendación: Utilizad minúsculas).
- NO se permiten los espacios en el nombre de elementos.
- SÍ se permiten las letras acentuadas (si las incluye el juego de caracteres especificado), los números y los signos: '**-**', '**:**', "**_**" y "**.**"
- La primera letra de una etiqueta ha de ser una letra, '**-**', '**:**', "**_**" o "**.**" , aunque se recomienda no utilizar '**:**'
- Tampoco se recomienda empezar por xml ó XML

El **ejemplar**, **raíz** o **elemento raíz** es el elemento aglutinador del resto.

Atributos

Los atributos son informaciones adicionales asociadas a los elementos. Se utilizan para "matizar" información sobre la información que guarda ese elemento, es decir, para distinguirlo de otro.

A diferencia del HTML, es obligatorio asignar un valor a los atributos. Valor del elemento

Ejemplo: 70

Contenido

El contenido es todo lo que están entre la etiqueta de apertura y de cierre.

- Se puede escribir texto
- No se puede escribir '', '<', '>', ' ' ', ' " ' para ello se utilizan las entidades (comienzan por &) predeterminadas: &;amp;, <, >, ', "
- Cualquier secuencia de espacios se considera un solo espacio
- También son espacios el retorno de carro y el tabulador
- Los elementos vacíos designan un elemento cuyo valor no ha sido definido: ó ¿Por qué un elemento vacío? Puede que el nodo ya aporte información al documento o que sea obligatorio para concordancia de versiones.
- Pueden ser más elementos hijos.

Analizador de documentos XML

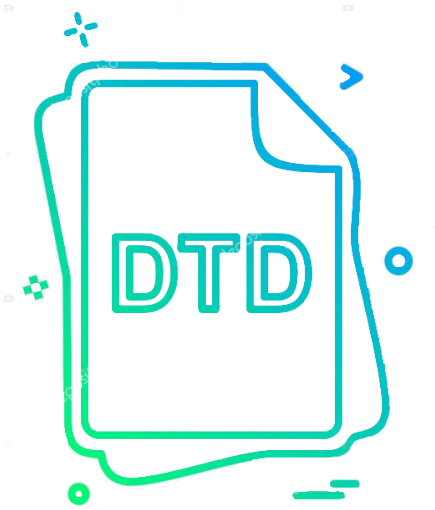
Antes de utilizar / procesar los documentos xml es importante analizarlo para comprobar que está correctamente construido, es decir esté BIEN FORMADO (escrito formalmente según la sintaxis y

reglas del XML). Será necesario utilizar un analizador de documentos XML (en inglés se llama parser) que puede ser:

- Cliente Web
- Comprobación vía Web: http://validator.w3.org ó http://www.xmlvalidation.com
- Software de edición XML:
    - XMLCopy Editor (Gratuito) [Recomendado para este curso]
    - Editix XML Editor (Tiene versión de prueba Gratuito).
    - Microsoft Core XML Services (MSXML) (Gratuito).
    - XMLFox Advance.
    - Altova XML Spy Edición Estándar.
    - Editor XML xmlBlueprint.
    - Editor Gráfico XSD y XML (XML Studio) (Gratuito).
    - Estudio XML Líquido (Gratuito).
    - Stylus Studio 2001 (Gratuito).
    - Oxygen XML Editor.
    - Exchanger XML Editor.

2.- Definiciones de tipo de documento, DTD.

Están formadas por una relación precisa de qué elementos pueden aparecer en un documento y dónde, así como el contenido y los atributos del mismo. Garantizan que los datos del documento XML cumplen las restricciones que se les haya impuesto en el DTD, ya que estas últimas permiten:



- Especificar la estructura del documento.
- Reflejar una restricción de integridad referencial mínima utilizando (ID e IDREF).
- Utilizar unos pequeños mecanismos de abstracción comparables a las macros, que son las entidades.
- Incluir documentos externos.

¿Cuáles son los inconvenientes de los DTD?

Los principales son:

- Su sintaxis no es XML.
- No soportan espacios de nombres.
- No definen tipos para los datos. Solo hay un tipo de elementos terminales, que son los datos textuales.
- No permite las secuencias no ordenadas.
- No es posible formar claves a partir de varios atributos o elementos.
- Una vez que se define un DTD no es posible añadir nuevos vocabularios.

Cuando están definidas dentro del documento XML se ubican entre corchetes después del nombre del ejemplar en el elemento pero, cuando está definido en un fichero externo ¿a qué tipo de fichero corresponde? Definimos el DTD externo en un fichero de texto plano con extensión **DTD**.

Autoevaluación

Marcar las afirmaciones referidas a un DTD:

- [ ] Permite la formación de identificadores compuestos por varios atributos.
- [ ] Para construirlas no se usa un lenguaje basado en XML.
- [ ] Permite incluir ficheros binarios en el documento.
- [ ] Definen distintos tipos para los datos.

Mostrar Información

2.1.- Declaración de Elementos

Los elementos se definirán por un nombre y una regla, a través de la etiqueta:



modelo_elemento podrá contener varios subgrupos, que a su vez, pueden contener otros elementos, datos textuales, nada o cualquier cosa.

Ejemplos:

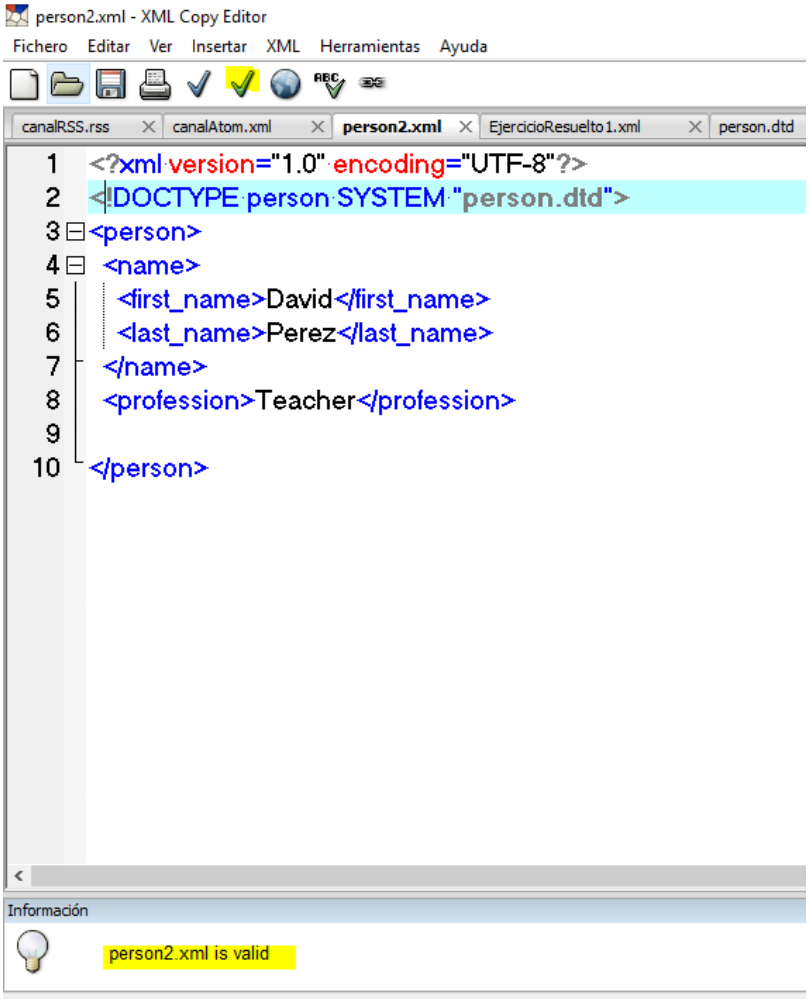
(A contiene al elemento y B y luego al C)
(A contiene datos textuales)
(A no contiene nada)
(A contiene datos textuales u otros elementos)

Si vamos a construir una DTD completa hay que recordar que el elemento raíz engloba al resto, por lo que la primera declaración de la DTD será siempre ese elemento raíz y entre paréntesis los elementos que hay dentro de él. Luego, en riguroso orden deberán aparecer la definición de dichos elementos. Si alguno de ellos tiene a su vez otros elementos hijos, se deben de definir en orden. Vamos con un ejemplo sencillo. ¿Cómo sería la DTD person.dtd que definiría el documento XML person2.xml mostrado anteriormente?

```
<!ELEMENT person (name, profession)>
<!ELEMENT name (first_name, last_name)>
<!ELEMENT first_name (#PCDATA)>
<!ELEMENT last_name  (#PCDATA)>
<!ELEMENT profession (#PCDATA)>
```

Ahora, en person2.xml podemos hacer dos cosas. Comprobar que está bien formado, y también que es válido. Es decir, que está construído según la descripción de la DTD a la que está vinculada, es decir person.dtd.

Si estás usando XMLCopyEditor sería con el “check verde”.



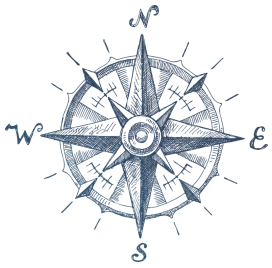
Autoevaluación

Los elementos terminales de tipo ANY son aquellos que están:

- ○ Formados por cadenas de texto exclusivamente.
- ○ Vacíos.
- ○ Formados por cualquier cosa.
- ○ Estará formada por otros elementos.

2.2.- Cardinalidad de elementos

¿Qué sucede cuando un elemento puede aparecer en el documento varias veces, hay que indicarlo de algún modo?



Pues sí, también hay que indicar cuando un elemento puede no aparecer. Para ello usamos los siguientes operadores, que nos permiten definir la cardinalidad de un elemento:

OPERADORES	DESCRIPCIÓN
Operador (?)	Define un componente opcional puede aparecer 0 ó 1 (está o no). En el siguiente ejemplo, el movil puede estar o no
Operador (+)	Define un componente presente al menos una vez (1, N) . En el siguienteVejemplo es el grupo de elementos codigopostal y ciudad los que al menos aparecerán una vez, pudiendo estar muchas veces (¡los dos!)
Operador (*)	Define un componente presente cero, una o varias veces (0,N):
Operador de elección |	Cuando se utiliza sustituyendo las comas en la declaración de grupos indica que para formar el documento XML hay que elegir entre los elementos separados por este operador. En el ejemplo siguiente, el documento XML tendrá elementos provincia que estarán formados por el elemento nombre y el cp (código postal), o por el elemento nombre y la ciudad, y como no hay operador de cardinalidad, sólo tendrá que salir una vez.

Ejercicio resuelto

Construye un documento XML bien formado y válido según la DTD pelicula2.dtd (Esta DTD sirve para definir documentos XML correspondientes a películas, con su director, título, actores, etc. Fíjate en las cardinalidades.Puedes probar a hacer varios)

```
<!ELEMENT pelicula (titulo, director, actores, duracion?)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT director (#PCDATA)>
<!ELEMENT actores (actor+)>
<!ELEMENT actor (#PCDATA)>
<!ELEMENT duracion (#PCDATA) >
```

Autoevaluación

La diferencia entre el operador ? y el + es que el primero permite que el elemento sobre el que se aplica esté presente una vez, como máximo, mientras que el operador + no limita el número máximo de veces que está presente el elemento en el documento XML.

- ○ Verdadero.
- ○ Falso.

2.3.- Atributos



Ya sabemos cómo declarar elementos, ahora veamos el modo de declarar los atributos asociados a un elemento. Los elementos pueden tener cero, uno o varios atributos. La **etiqueta ATTLIST** permite declarar una lista de atributos para un elemento especificado (se puede llamar una vez y definir varios elementos o llamar varias veces a ATTLIST).

Los **atributos** pueden ser **opcionales** #IMPLIED u **obligatorios** #REQUIRED y en ocasiones **valores fijos** #FIXED.

Se puede definir un valor de atributo por defecto “valor_predeterminado”

Los atributos pueden ser de diferentes tipos. Veamos los más significativos:

OPERADORES	DESCRIPCIÓN
CDATA	Es el tipo de atributo para los datos textuales. ……. Jim Morrison ……. Love will tear us apart
Enumeración	Es una lista de todos los valores posibles para el atributo separados por barras verticales. Si el XML tuviera un valor diferente, daría un error de validación. …….. Madonna …… Madonna …….. Metallica

ID

Permite declarar un atributo identificador en un elemento. Hay que recordar que este valor ha de ser único en el documento (no lo puede tomar otro atributo). Además hay que tener en cuenta que los números (ni un literal que empiece por número) no son nombres válidos en XML, por tanto no son un identificador legal de XML. Para resolverlo suele incluirse un prefijo en los valores y separarlo con un guión o una letra.

```
.....
David
.....
David
```

IDREF

Permite hacer referencias a identificadores. En este caso el valor del atributo ha de corresponder con el de un identificador de un elemento existente en el documento.

NMTOKEN

Permite determinar que el valor de un atributo ha de ser una sola palabra compuesta por los caracteres permitidos por XML, es decir, que siga las normas con las que nombramos a un elemento XML. Esta opción limita y acota los valores posibles. Está enfocado a limitar la validez de los documentos XML y que éstos se adecúen mejor a la DTD “exigida”.

No evita que year pueda ser “99” ó “Marzo” pero sí algún error como "1 990".

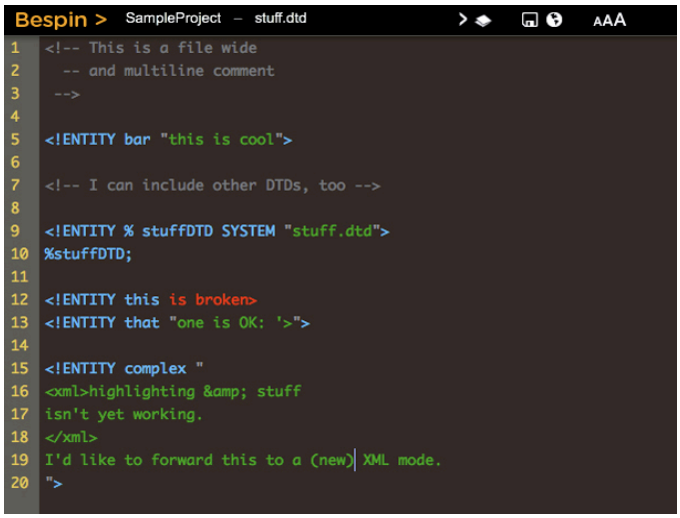
Autoevaluación

¿Cuál de los siguientes atributos permite añadir a los datos una restricción de integridad referencial?

- ○ MTOKEN.
- ○ ID.
- ○ IDREF.
- ○ CDATA.

2.4.- Declaraciones de entidades.

¿Qué sucede si queremos declarar valores constantes dentro de los documentos? ¿podemos?
Las **entidades** nos permiten definir constantes en un documento XML.



Además de las **entidades predeterminadas** (⩽ > " ' & que corresponden con los símbolos: <, >, “, ‘ y &), podremos crear las nuestras propias. ¿Qué podremos hacer con las entidades?

- Inserción de caracteres especiales.
- Simplificación de textos largos.
- División modular de los documentos.

Cuando se usan dentro del documento XML se limitan por "&" y ";", por ejemplo &entidad; ¿Cómo trabaja el intérprete con ellos? Al procesar el documento XML, el intérprete sustituye la entidad por el valor que se le ha asociado en el DTD.

No admiten recursividad, es decir, una entidad no puede hacer referencia a ella misma.
Para definir una entidad en un DTD se usa el elemento < ! ENTITY>

Vamos a ver algunos ejemplos de uso, aunque el uso avanzado de entidades queda fuera de los objetivos de este curso:

OPERADORES	Ejemplos
Inserción de caracteres especiales	(Parecido a la definición de constantes en programación). Por ejemplo, insertar el carácter del € = #x20ac se haría 19,95 € Pero se puede declarar una entidad como esta y luego utilizarse en el documento XML [dentro de la DTD] …… 19,95 € [dentro del documento XML que define la DTD anterior]
Textos que se repiten ahorrará texto que escribir	Por ejemplo opinión sobre un artista: [dentro de la DTD] …… Ciudadano Kane &om;

[dentro del documento XML que define la DTD anterior]

Ejercicio de refuerzo...

Escribe varios documentos XML (bien formados) que estén descritos por la siguiente DTD: person2.dtd y que lo vinculen de manera correcta (sean válidos). (person.dtd)

estadoCivil (Soltero | Casado | Divorciado) #IMPLIED>

Construir una DTD y posteriormente un documento XML con varios elementos libro (bien formado y válido con respecto a dicha DTD). Dicha DTD debe representar una librería de la siguiente forma:

- Libreria: elemento raíz compuesto por libros.
- Libro: productos de la librería.
- Título: un libro.
- Categoría: lista de categorías de libros (novela, poesía, teatro, ensayo). Mejor como atributo de título.
- Precio: precio de un libro.
- Autor: autor del libro.
- ISBN: código del libro.
- Editor: editor del libro
- Cantidad: número de productos disponibles en la tienda.

Puedes ver el resultado del ejercicio descargando "ejerciciosresueltosdtd.zip"

Para saber más...

Además de lo visto hasta el momento, las DTD permiten incluir o excluir partes de la DTD de cara a las validaciones, además de incorpoar otro tipo de delcaraciones de notificación. Se harían con palabras clave como IGNORE, INCLUDE o NOTATION. Consideramos que estos aspectos son de uso muy avanzado que quedan fuera del

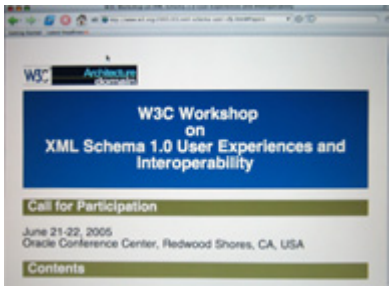
alcance de este curso. Dejamos algunos links donde se puede encontrar más info sobre ello.

Autoevaluación

Las entidades permiten definir elementos cuyo valor es constante dentro de un DTD.

- ○ Verdadero.
- ○ Falso.

3.- XML Schema.



Los **DTD** permiten diseñar un **vocabulario para ficheros XML**, pero, ¿qué sucede cuando los valores de los elementos y atributos de esos ficheros han de corresponder a datos de un tipo determinado, o cumplir determinadas restricciones que no pueden reflejarse en los DTD? Para ello se definen **XML Schemas**.

¿También **se definen en ficheros planos**? Si, ya que son documentos XML, pero en este caso la extensión de los archivos es xsd, motivo por el cual también se les denomina documentos XSD.

Los elementos XML que se utilizan para generar un esquema han de pertenecer al espacio de nombre XML Schema, que es: http://www.w3.org/2001/XMLSchema.

El ejemplar de estos ficheros es , contiene declaraciones para todos los elementos y atributos que puedan aparecer en un documento XML asociado válido. Los elementos hijos inmediatos de este ejemplar son que nos permiten crear globalmente un elemento. Esto significa que el elemento creado puede ser el ejemplar del documento XML asociado.

Ejercicio resuelto

Creación de un esquema correspondiente al siguiente documento XML:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!--documento xmlejercicioresuelto.xml-->
<!DOCTYPE alumno>

<alumno xmlns:xs="http://www.w3.org/2001/XMLSchema-instance" xs:noNamespaceSchemaLocati
Olga Velarde Cobo
</alumno>
```

Debes conocer...

En este primer enlace encontrarás los fundamentos del estándar XML Schema.

XML Schema Fundamentos

3.1.- Limitaciones de los DTD: necesidades XML Schema

Crear documentos XML bien formados garantiza que cualquier analizador que se atenga a las normas será capaz de cargarlo y trabajar con él adecuadamente y también todas las personas que trabajen en ello. Con las DTDs se logra mantener la uniformidad de los documentos, pero las DTDs tienen limitaciones:



- **No se puede restringir el tipo de datos que pueden tomar parte** (todo texto). Por ejemplo, permitiría un literal en un campo que supuestamente ha de ser numérico.
- **No siguen las normas de los documentos XML**. Una DTD externa no es un documento XML válido, lo cual pone trabas a la automatización de procesos.

Estas razones, entre otras, hacen que se necesite otra herramienta para definición de documentos XML que son los **XML Schema** (o esquema).

Algunas diferencias entre DTD y Esquemas:

- Un **esquema** sí que es un documento XML (seguirán las normas de los documentos XML). Utilizar el mismo formato para crear un documento y su descripción nos facilitará las tareas.
- Los esquemas son procesables.
- Los esquemas son más potentes. Es posible describir con mayor detalle y restricciones un documento XML utilizando esquemas que utilizando DTD (podremos utilizar tipos de datos).
- Los esquemas pueden implantar reglas mucho más específicas que las DTD sobre el contenido de los elementos y atributos.
- No se puede describir entidades utilizando esquemas. Necesitaremos utilizar DTDs.

3.2.- Facetas de los tipos de datos.

XML Schema es una especificación del W3C de 2001. Puedes acceder a ella en el siguiente

link: http://www.w3.org/XML/Schema



Los Esquemas serán documentos XML en un archivo aparte con extensión .xsd
Un esquema XML define la estructura válida para un tipo de documento XML (al igual que las DTD), es decir:

- Los elementos que pueden aparecer en el documento
- Los atributos que pueden utilizarse junto a cada elemento
- Cómo se pueden anidar los elementos (padres e hijos)
- El orden en el que deben aparecer los elementos hijos de un mismo padre
- El número permitido de elementos hijos
- Si un elemento puede ser vacío o no
- Tipos de datos para elementos y atributos
- Valores por defecto y fijos para elementos y atributos

En el documento XML que se define según el Esquema, incluiremos una referencia al XSD.

3.2.1.- Espacios de nombre o namespace

Existe libertad para asignar nombres a las etiquetas que utilizamos en nuestros documentos XML, siempre que sigan las normas sintácticas que ya explicamos. Al aumentar el número de aplicaciones que trabajan con el XML van a existir etiquetas duplicadas, lo que generaría un conflicto para documentos públicos. Los espacios de nombres existen para solucionar este tipo de problemas, un espacio de nombre o namespace es un prefijo que sirve para diferenciar unas etiquetas de otras.

Los Esquemas XML utilizan espacios de nombre internamente. Los más habituales son xs: ó xsd: Los elementos XML que se utilizan para generar un esquema han de pertenecer al espacio de nombre XML Schema, que es: http://www.w3.org/2001/XMLSchema. En la práctica, esos namespaces serán prefijos que se pondrán delante de los elementos y tipos.

3.2.2.- ¿Cómo vincular un esquema a un documento XML?

El documento XML debe referenciar al espacio de nombre de instancia de los esquemas para poder utilizar atributos especiales que ayudan a vincularlo con el esquema:

ATRIBUOS ESPECIALES	DESCRIPCIÓN
xsi: schemaLocation	En un elemento contiene una lista de espacio de nombre usados dentro de dicho elemento y los URL de los esquemas con los que se validan los elementos y atributos en dicho espacio de nombre.
xsi: **noNamespaceSchemaLocation**	Contiene un URL para el esquema usado para validar elementos que no se encuentran en ningún espacio de nombres.

Esquema asociado a espacio de nombres:

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation=
“esquema.xsd”>
```

Esquema NO asociado a espacio de nombres

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation= “esquema.xsd”>
```

3.3.- Construcción de un Esquema



Un Esquema es un documento XML, por lo que la primera línea será el prólogo que define la versión del XML y los códigos de caracteres utilizados.

La **raíz** del documento de todo Esquema debe ser un elemento schema, que en su forma más sencilla será:

El atributo xmlns nos permitirá indicar la norma que seguirá el esquema. Habrá una etiqueta de cierre una vez que se hayan definido todos los elementos y la norma que éstos deben seguir en cualquier documento XML.

Detallaremos cómo se construyen los esquemas.

Debes conocer...

En este primer enlace encontrarás los fundamentos del estándar XML Schema

3.3.1- Definición de Elementos Simples y sus tipos de datos.

Definición de Elementos Simples.

Los elementos simples pueden contener cualquier tipo de dato pero no atributos ni otros elementos hijos.

Para definir un elemento simple, utilizamos la sintaxis:

Un elemento simple puede tener un valor por defecto y un valor “fijo”. Esto se indica mediante los atributos default y fixed

Tipos de datos de Elementos simples.

Los tipos de datos impiden que se introduzcan valores incorrectos entre las etiquetas de elementos del documento XML. Pueden ser predefinidos o construidos por el usuario. De los predefinidos, los más frecuentes son:

- string = cadena de texto, ejemplo=“Brazil”
- boolean = valores booleanos: true / false
- byte, short, integer, long = nºs decimales, ejemplo= -7, 0, 12.
- float, double, decimal =nºs decimales, -47,1, 1766,86
- date = fecha AAAA-MM-DD, ejemplo= 1999-11-23
- time = hora HH:MM, ejemplo= 23:03

Ejemplos:

Vamos a ver un documento XML válido muy simple (peliculaxsd.xml), de un único elemento, y el esquema que lo definiría (peli_esquema.xsd)

```
<?xml version="1.0" encoding="UTF-8"?>
<pelicula xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation= "peli_esquema.xsd">
Brazil
</pelicula>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="pelicula" type="xs:string" />
</xs:schema>
```

Descargar el código peliculaxsd.xml y peli_esquema.xsd

Debes conocer...

En este enlace encontrarás los tipos de datos admitidos por el estándar.

XML Schema Data Types

3.3.2.- Grupos de elementos

Como comprendereis, no se pueden definir muchos documentos XML con un único tipo simple, por lo que se necesitan declarar grupos de elementos. Para añadir más elementos a un esquema vamos a utilizar otras etiquetas. La primera es todos los elementos irán entre esas etiquetas, indicando que hay un grupo de elementos o un único elemento con atributos.

A su vez los grupos podrán estar definidos entre otra etiqueta que será una de estas, dependiendo de cómo lo queríamos definir.

- han de incluirse todos los elementos, sin importar el orden.
- han de incluirse todos los elementos respetando el orden.
- Es necesario incluir uno de los elementos.

Ejemplos:

```
<?xml version="1.0"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema">
 <element name="pelicula">
  <complexType>
   <sequence>
    <element name="titulo" type="string" />
    <element name="minutos" type="integer" />
    <element name="fecha_estreno" type="date" />
   </sequence>
  </complexType>
 </element>
 </schema>

<?xml version="1.0" encoding="UTF-8"?>
<pelicula xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation= "peliculact.xsd">
 <titulo>Torrente</titulo>
 <minutos>90</minutos>
 <fecha_estreno>1998-04-04</fecha_estreno>

</pelicula>
```

Descarga el código peliculactxml.xml y peliculact.xsd

En el ejemplo no hemos puesto namesespaces con el prefijo xs para que veais que también se podría definir así. Prueba a cambiar los valores de minutos y fecha_estreno por otros de diferente tipo para comprobar que no validarían.

Ejercicio de refuerzo

Crea un Esquema.xsd con elementos simples que sirva para definir un coche. El elemento principal sería coche y luego tendría, marca, modelo, color, matrícula (numérico), año (fecha), kms (numérico). Después, construye un documento XML según ese esquema, vincula y valídalos.

Autoevaluación

Para hacer un elemento complejo formado por un listado de elementos en los que importa el orden hay que usar el elemento:

- ○
- ○
- ○
- ○

3.3.3.- Cardinalidad

Con lo visto hasta ahora no podremos repetir un elemento. Necesitamos definir los parámetros de cardinalidad.

Para documentos XML en los que necesitemos definir un elemento que se repita, podremos utilizar los atributos de element, **maxOccurs** (controlará el número de apariciones máximas del elemento) y **minOccurs** (controlará las apariciones mínimas del elemento). Por defecto, estos valores se toman siempre como 1. Es decir, cuando se define un elemento como máximo y como mínimo tendrá que aparecer una vez. Cuando **maxOccurs="unbounded"** puede aparecer tantas veces como se quiera. minOcurrs controlará las apariciones del elemento.

Estos atributos no pueden aplicarse al elemento raiz.

Ejemplos:

En este ejemplo se define un elemento persona que tiene la posibilidad de definir un apellido o dos. ¿Te animas a construir un documento XML válido para este esquema?

```
<?xml version="1.0"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema">
 <element name="persona">
  <complexType>
   <sequence>
    <element name="nombre" type="string"/>
    <element name="apellido" type="string" minOccurs="1" maxOccurs="2"/>
    <element name="edad" type="integer"/>
   </sequence>
  </complexType>
  </element>
 </schema>
```

Ejercicio resuelto

¿Cómo haríamos un esquema que permitiera repetir tantos elementos personas como se quisiera?Es decir: que fuera como esta DTD.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="personas">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="persona" minOccurs="1" maxOccurs="unbounded">
          <xs:complexType>
           <xs:sequence>
            <xs:element name="nombre" type="xs:string"/>
            <xs:element name="apellido" type="xs:string" minOccurs="1" maxOccurs="2"/>
            <xs:element name="edad" type="xs:integer"/>
           </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

3.4.4.- Atributos y Facetas o Restricciones

Atributos

Podremos definir atributos como elementos simples utilizando attribute en lugar de element. Los tipos de datos serán los mismos. Como hacíamos en las DTDs, estos atributos podrán ser opcionales u obligatorios, por lo que habrá que definirlos como use="optional" o use="required".

Ejemplo 1:

Ejemplo 2:

Definición de un elemento persona con varios elementos hijos y un atributo.

```
type="string"/>
type="string"/>
……
David
Perez
42
```

Facetas o Restricciones

Uno de los aspectos más potentes de los esquemas no es sólo definir tipos para los datos, sino también introducir restricciones para los posibles valores del contenido de dichos elementos y atributos. Eso se consigue con las facetas o Restricciones que modificarán la manera de definir el tipo de un elemento, necesitando dos etiquetas más: simpleType y restriction.

Existen diferentes tipos de facetas:

- Valor comprendido en un rango

  enumeration: Establece una lista de valores "aceptados"

Ejemplo:

Aquí se define un elemento car que solo podrá tener los valores Audi, Golf y BMW.

- El valor está restringido a un conjunto de valores posibles

  maxExclusive y maxInclusive: Valor máximo de un rango

  minExclusive y minInclusive: Valor mínimo en un rango

  Ejemplo:

- Longitud de los valores de los elementos

  Para tipo string:

  length: Número de caracteres obligatorios

  maxLength y minLength: Número máximo y mínimo de caracteres permitidos

  Para tipos numéricos:

  totalDigits: Número exacto de dígitos permitidos

  fractionDigits: Número máximo de posiciones decimales permitidas

  Ejemplo:

  Se define un elemento password que sólo admitirá un literal de 8 caracteres.

- Restringir el valor de un elemento a una serie de caracteres, lo que se conoce como patrones o pattern, expresiones regulares, etc. Existen muchísimas opciones y variaciones.

  pattern: Define una secuencia de caracteres permitida como un patrón determinado (se establece un patrón regular que establece dónde puede aparecer un tipo determinado de caracteres). Se definirá como

  - . Cualquier carácter
  - \d cualquier dígito
  - \D cualquier no-dígito

- \s espacio en blanco, retorno de carro, línea nueva
- \S cualquier carácter distinto a espacio en blanco
- (ab)* cualquier cosa entre paréntesis puede aparecer cero o más veces
- (ab)+cualquier cosa entre paréntesis puede aparecer una o más veces
- a{n} a puede aparecer en una cadena n veces
- [a] uno de los caracteres que aparezcan entre []

Ejemplos:

En este ejemplo, el elemento “letter” debe tomar como valor 1 letra minúscula (sólo 1)

En este ejemplo, el elemento “initials” debe tomar como valor 3 letras mayúsculas o minúscula (sólo 3)

En este ejemplo, el elemento “option” debe tomar como valor una de estas letras: x, y o z

En este ejemplo, el elemento “proid” debe tomar un valor con cinco dígitos del 0 al 9.

En este ejemplo, el valor del campo “password” debe ser 8 caracteres que sean letras mayúsculas o minúsculas y dígitos del 0 al 9.

Ejercicio resuelto

Construye un documento XML bien formado y con contenido válido según el esquema *AccionesFormativas3.xsd*

Para saber más...

Hasta aquí hemos visto lo más usado en la definición de esquemas, existen muchas más opciones y parámetros, la posibilidad de definir tipos propios complejos, definir contenidos, elementos vacíos, etc. Puedes ampliar en el Tutorial del W3CSchool sobre XSD.

